

# SOAP

SYSTEM INTEGRATION

Arturo Mora-Rioja  
[amri@ek.dk](mailto:amri@ek.dk)

**EK**



# SOAP – Simple Object Access Protocol

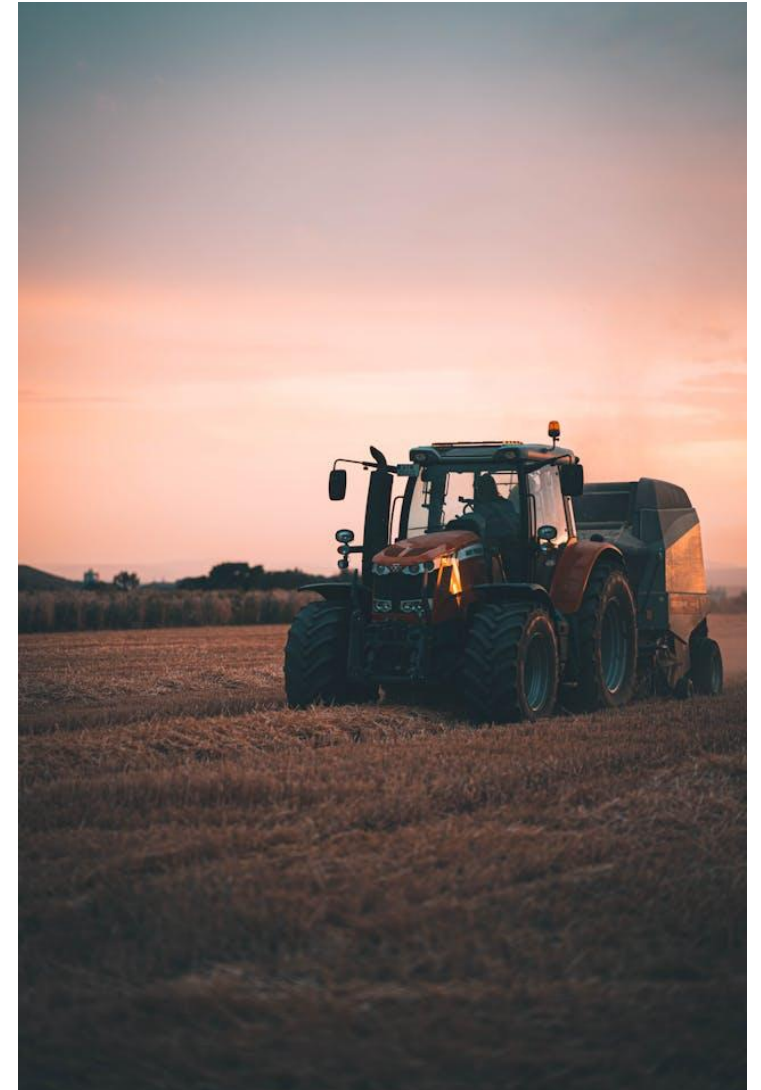
- [W3C message specification](#) for information interchange between systems
  - Defined as XML-RPC (1998). Developed by UserLand, DevelopMentor and Microsoft (1999)
  - Very structured and formal
  - Not an architectural style, but a messaging protocol
    - It defines the rules for a web service to perform operations
      - It describes the message format and specifies how to perform CRUD operations
      - It does not define the service that processes the information or the operations
  - Stateless, yet made stateful by its WS-extensions (e.g., WS-Security)
- Transport media: HTTP/HTTPS, SMTP, JMS, message queues (RabbitMQ, IBM MQ)
  - In HTTP/HTTPS, the specification focuses on POST, but GET can be technically used

# SOAP

- Message file type: XML
  - Based on a WSDL specification
- SOAP messages can carry attachments
  - Transmission of binary data alongside XML content
- Processes addressing SOAP messages must provide a SOAP processor that parses XML
- Used mainly for distributed enterprise environments
  - Banking applications (e.g., money transfers, usually involving different bank branches)
  - Insurance companies (distributed information across geographically dispersed offices)
  - Flight booking applications (secure interconnection of diverse systems)
  - City management (e.g., traffic light management, sewage system operations)

# SOAP Advantages and Disadvantages

- 👍
  - Reliable
  - Secure. It has built-in error handling
  - Heavily standardised
  - Transport-, platform- and language-agnostic
- 👎
  - Slow
  - Complicated
  - Not flexible
  - Does not support caching



© [Volker Thimm](#) @ Pexels

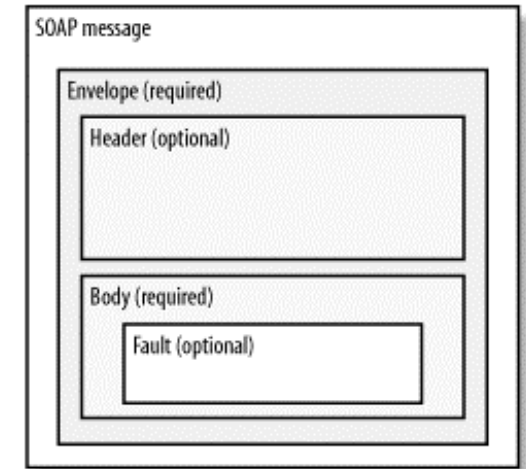
# Companies and Services That Use SOAP

- SWIFT. Global interbank messaging. SEPA/EPC banking services. EU transfers
- Visa and Mastercard merchant services. They combine SOAP with REST endpoints
- Bloomberg Enterprise Data Services. Market data delivery
- Danske Bank, Nordea, Banco Santander, BNP Paribas, Deutsche Bank, ING, Barclays, Chase
- National eHealth platforms in Denmark, Sweden, Spain, UK. Labs, prescriptions, referrals
- Danish government platforms: Skat, BBR, ATP, CPR, Statistics Denmark, NemID
  - [Digitaliseringsstyrelsen's SOAP profile for Danish public-sector web services](#)
- SAP ERP, Salesforce Enterprise, Microsoft Dynamics CRM, Oracle E-Business Suite
- Airline reservation systems: Amadeus, Sabre, Galileo. Booking
- Logistics: FedEx, UPS, DHL. Shipment tracking and label creation
- Phone provisioning systems: Vodafone, AT&T, Deutsche Telekom, Orange, Telia, TDC
- Payment gateways: PayPal, eBay

# SOAP Message

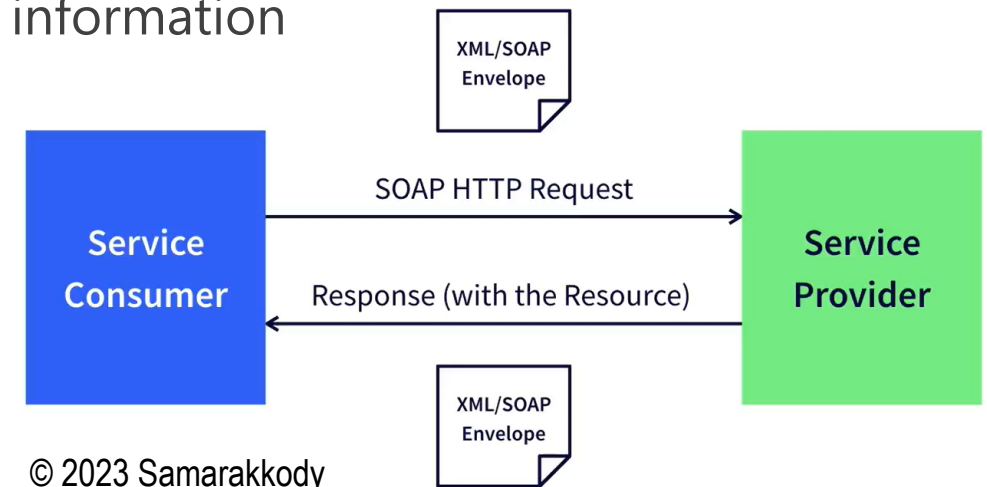
- **Envelope**

- Defines the structure of the message
- One per message
- Its version is defined by an XML namespace
  - SOAP 1.1: <https://schemas.xmlsoap.org/soap/envelope/>
  - SOAP 1.2: <https://www.w3.org/2001/09/soap-envelope>



© 2002 Cerami

- **Header** (optional). Authentication and authorisation information
- **Body**. Usually includes requests and responses
  - **Fault**. Present in case of error
- **Encoding**. Rules for defining the data types
  - Not mandatory, but recommended



© 2023 Samarakkody

# SOAP Example

- ISBN book validation service

- Request

```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <IsValidISBN10 xmlns="http://webservices.daehosting.com/ISBN">
      <sISBN>0-19-852663-6</sISBN>
    </IsValidISBN10>
  </soap:Body>
</soap:Envelope>
```

- Response

```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <m:IsValidISBN10Response xmlns:m="http://webservices.daehosting.com/ISBN">
      <m:IsValidISBN10Result>true</m:IsValidISBN10Result>
    </m:IsValidISBN10Response>
  </soap:Body>
</soap:Envelope>
```

# SOAP Error Example

- The client has requested a method `ValidateCreditCard` that does not exist in the server
- The server answers with this response message:

```
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <SOAP-ENV:Body>
    <SOAP-ENV:Fault>
      <faultcode xsi:type="xsd:string">SOAP-ENV:Client</faultcode>
      <faultstring xsi:type="xsd:string">
        Failed to locate method (ValidateCreditCard) in class
        (examplesCreditCard) at /usr/local/ActivePerl-5.6/lib/
        site_perl/5.6.0/SOAP/Lite.pm line 1555.
      </faultstring>
    </SOAP-ENV:Fault>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

# SOAP Encoding

- **Scalar data types**

- string
- Boolean
- binary
- float, double, decimal
- integer,  
positiveInteger, nonPositiveInteger, negativeInteger, nonNegativeInteger,  
byte, short, int, long,  
unsignedByte, unsignedShort, unsignedInt, unsignedLong,
- date, time

- Example

```
<return xsi:type="xsd:decimal">54.99</return>
```

# SOAP Encoding

- **Compound data types**
  - User-defined (e.g., a purchase order, a list of stock quotes)
- Example (array)

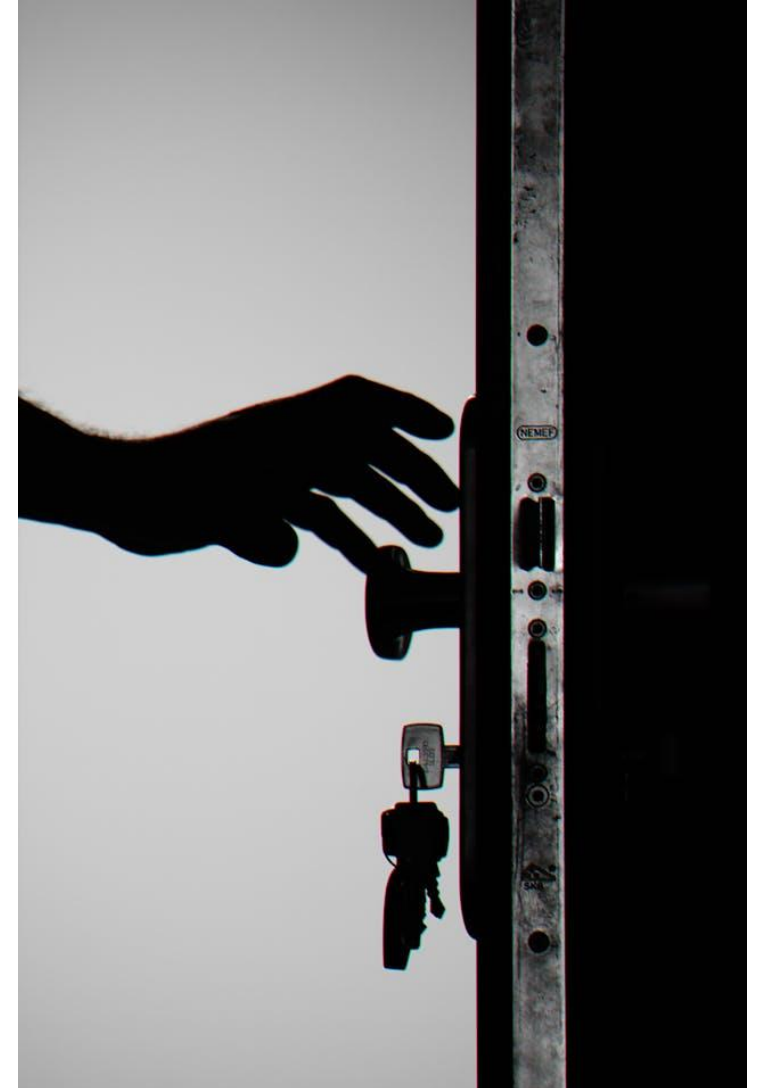
```
<return
  xmlns:ns2="http://www.w3.org/2001/09/soap-encoding"
  xsi:type="ns2:Array" ns2:arrayType="xsd:decimal[2]">
  <item xsi:type="xsd:decimal">54.99</item>
  <item xsi:type="xsd:decimal">19.99</item>
</return>
```

- Example (construct)

```
<return xmlns:ns2="urn:examples" xsi:type="ns2:product">
  <name xsi:type="xsd:string">Red Hat Linux</name>
  <price xsi:type="xsd:decimal">54.99</price>
  <description xsi:type="xsd:string">
    Red Hat Linux Operating System
  </description>
  <SKU xsi:type="xsd:string">A358185</SKU>
</return>
```

# WS-Security

- Standard extension for SOAP to provide message level security
  - Authentication
    - It can use UsernameToken, X.509 certificates, SAML (Security Assertion Markup Language)
  - Integrity
    - Through digital signatures
  - Confidentiality
    - Via symmetric and asymmetric encryption algorithms
  - Non-repudiation



© [George Becker](#) @ Pexels



# WS-Security Example

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:wss="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd"
  xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd"
  xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
  xmlns:xenc="http://www.w3.org/2001/04/xmenc#">
  <soap:Header>
    <wsse:Security soap:mustUnderstand="1">
      <!-- Authentication (username and password) -->
      <wsse:UsernameToken wsu:Id="UsernameToken-1">
        <wsse:Username>alice</wsse:Username>
        <wsse:Password
          Type="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-username-token-profile-1.0#PasswordText">
            secret
          </wsse:Password>
        </wsse:UsernameToken>

        <!-- Certificate used for signature / encryption. It includes the sender's public key -->
        <wsse:BinarySecurityToken
          wsu:Id="X509Token"
          ValueType="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-x509-token-profile-1.0#X509v3"
          EncodingType="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-soap-message-security-1.0#Base64Binary">
            MIID...base64-certificate-data...
          </wsse:BinarySecurityToken>
```

If the receiver cannot process a tag marked with  
mustUnderstand="1" or mustUnderstand="true",  
the message must be rejected and a SOAP fault returned

# WS-Security Example (cont.)

```
<!-- Integrity: digital signature over the Body -->
```

```
<ds:Signature>
```

```
  <ds:SignedInfo>
```

```
    <ds:CanonicalizationMethod Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" />
```

```
    <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256" />
```

```
    <ds:Reference URI="#Body-1">
```

```
      <ds:Transforms>
```

```
        <ds:Transform Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" />
```

```
      </ds:Transforms>
```

```
      <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#sha256" />
```

```
      <ds:DigestValue>...</ds:DigestValue>
```

```
    </ds:Reference>
```

```
  </ds:SignedInfo>
```

```
  <ds:SignatureValue>...</ds:SignatureValue>
```

```
  <ds:KeyInfo>
```

```
    <wsse:SecurityTokenReference>
```

```
      <wsse:Reference
```

```
        URI="#X509Token"
```

```
        ValueType="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-x509-token-profile-1.0#X509v3" />
```

```
    </wsse:SecurityTokenReference>
```

```
  </ds:KeyInfo>
```

```
</ds:Signature>
```

Reference to the body by ID.

Canonicalization: standardisation of extra spaces, line breaks, namespaces, character encoding, etc.

RSA signature over SignedInfo.

Pointer to the previously defined certificate.

# WS-Security Example (cont.)

```
<!-- Confidentiality: encrypted symmetric key used to decrypt the body -->
<xenc:EncryptedKey>
  <xenc:EncryptionMethod Algorithm="http://www.w3.org/2001/04/xmlenc#rsa-oaep-mgf1p"/>
  <ds:KeyInfo>
    <wsse:SecurityTokenReference>
      <wsse:Reference
        URI="#X509Token"
        ValueType="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-x509-token-profile-1.0#X509v3"/>
    </wsse:SecurityTokenReference>
  </ds:KeyInfo>
  <xenc:CipherData>
    <xenc:CipherValue>...</xenc:CipherValue>
  </xenc:CipherData>
  <xenc:ReferenceList>
    <xenc:DataReference URI="#EncBody-1"/>
  </xenc:ReferenceList>
</xenc:EncryptedKey>
</wsse:Security>
</soap:Header>

<soap:Body wsu:Id="Body-1">
  <!-- Confidentiality: encrypted payload -->
  <xenc:EncryptedData
    Id="EncBody-1"
    Type="http://www.w3.org/2001/04/xmlenc#Content">
    <xenc:EncryptionMethod Algorithm="http://www.w3.org/2001/04/xmlenc#aes256-cbc"/>
    <xenc:CipherData>
      <xenc:CipherValue>...</xenc:CipherValue>
    </xenc:CipherData>
  </xenc:EncryptedData>
</soap:Body>
</soap:Envelope>
```

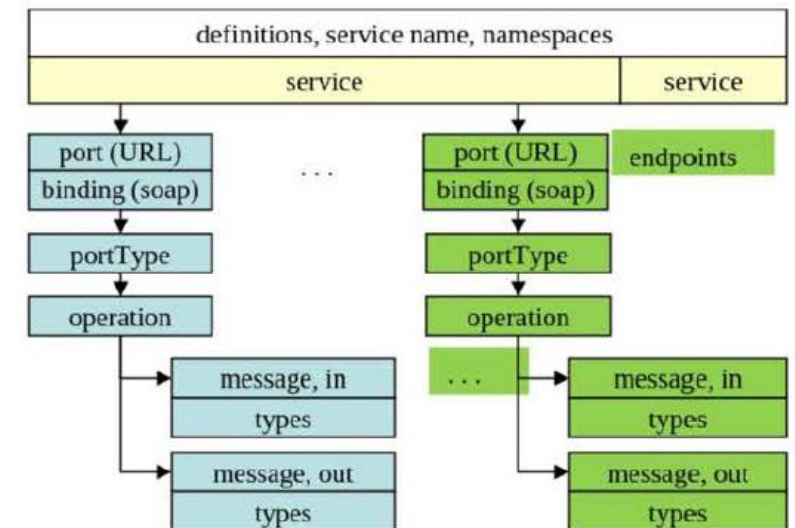
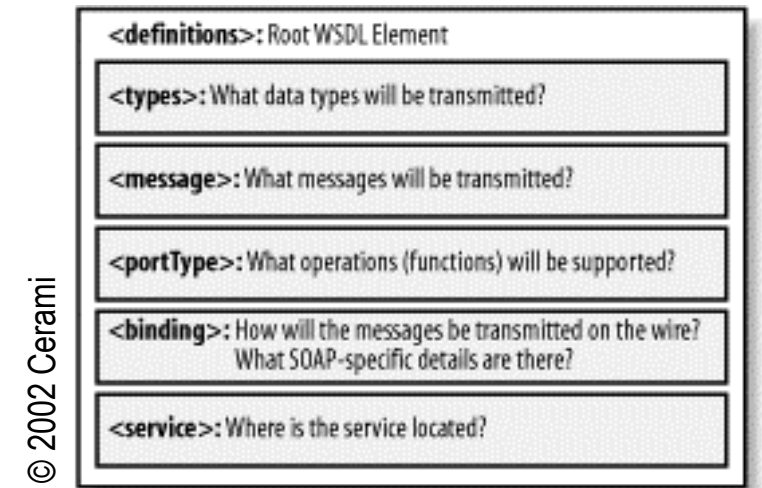
Symmetric AES key encrypted with the receiver's public key.

Points to the encrypted body element.

Encrypted payload.

# WSDL – Web Services Description Language

- XML specification that describes the interface of a web service
- It serves as a contract for both client (service consumer) and server (service provider)
- It defines
  - The available operations
  - The messages (requests and responses)
  - The XML data types (via XSD)
  - The transport and binding details (e.g., SOAP over HTTP)
  - The endpoint address of the service
- Both client and server generate code from it
- UDDI – Universal Description, Discovery, and Integration
  - Publishing and discovery protocol. Now obsolete



# WSDL Example

Root element. It defines:

- The name of the web service
- The namespaces used in the document

Target namespace. Defined by the user.

It describes a one-way message (request or response).  
Its parts can be request parameters or return values.  
There can be several parts per message.

It can define multiple operations,  
each one usually combining a request (input message)  
and a response (output message).

An operation can have an `<xsd:fault>` element  
to encapsulate errors.

There usually is a `<wsdl:types>` section  
specifying compound data types to use  
for parameters and return values.

By default they are defined in XSD format.

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions name="HelloService"
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
  targetNamespace="http://www.ecerami.com/wsdl/HelloService.wsdl"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns="http://www.ecerami.com/wsdl/HelloService.wsdl"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <wsdl:message name="SayHelloRequest">
    <part name="firstName" type="xsd:string"/>
  </wsdl:message>
  <wsdl:message name="SayHelloResponse">
    <part name="greeting" type="xsd:string"/>
  </wsdl:message>
  <wsdl:portType name="HelloPortType">
    <wsdl:operation name="sayHello">
      <wsdl:input message="tns:SayHelloRequest"/>
      <wsdl:output message="tns:SayHelloResponse"/>
    </wsdl:operation>
  </wsdl:portType>
  ...
</wsdl:definitions>
```

Messages qualified in  
the target namespace.

# WSDL Example (cont.)

Could also specify SMTP, JMS, SOAP-over-UDP or custom MQ.

It specifies how the service will be implemented, usually via an HTTP connection.

Instead of `<soap:binding>` it could be `<http:binding verb="GET">` or `<http:binding verb="POST">`.

It defines the address for invoking the service. Usually a URL.

There can also be a `<wsdl:import>` section to import other WSDLs or XSDs.

```
...
<wsdl:binding name="HelloBinding" type="tns:HelloPortType">
  <soap:binding style="document"
    transport="http://schemas.xmlsoap.org/soap/http"/>
  <wsdl:operation name="sayHello">
    <soap:operation
      soapAction="http://www.ecerami.com/wsdl/HelloService.wsdl/SayHello"/>
    <wsdl:input>
      <soap:body use="literal"/>
    </wsdl:input>
    <wsdl:output>
      <soap:body use="literal"/>
    </wsdl:output>
  </wsdl:operation>
</wsdl:binding>
<wsdl:service name="HelloService">
  <documentation>WSDL File for HelloService</documentation>
  <wsdl:port binding="tns:HelloBinding" name="HelloPort">
    <soap:address
      location="https://api.example.com/hello"/>
  </wsdl:port>
</wsdl:service>
</wsdl:definitions>
```

Based on © 2002 Cerami

# WSDL Operation Types

- **One-way.** The service receives a message

```
<wsdl:operation name="NotifyAddressChange">  
  <wsdl:input message="tns:NotifyAddressChangeInput"/>  
</wsdl:operation>
```

- **Request-response.** The service receives a message and sends a response

```
<wsdl:operation name="SubmitFeedback">  
  <wsdl:input message="tns:SubmitFeedbackInput"/>  
  <wsdl:output message="tns:SubmitFeedbackOutput"/>  
</wsdl:operation>
```

- **Solicit-response.** The service sends a message and receives a response

```
<wsdl:operation name="SolicitConsent">  
  <wsdl:output message="tns:ConsentSolicitation"/>  
  <wsdl:input message="tns:ConsentReply"/>  
</wsdl:operation>
```

- **Notification.** The service sends a message

```
<wsdl:operation name="PublishCourseUpdate">  
  <wsdl:output message="tns:PublishCourseUpdateOutput"/>  
</wsdl:operation>
```

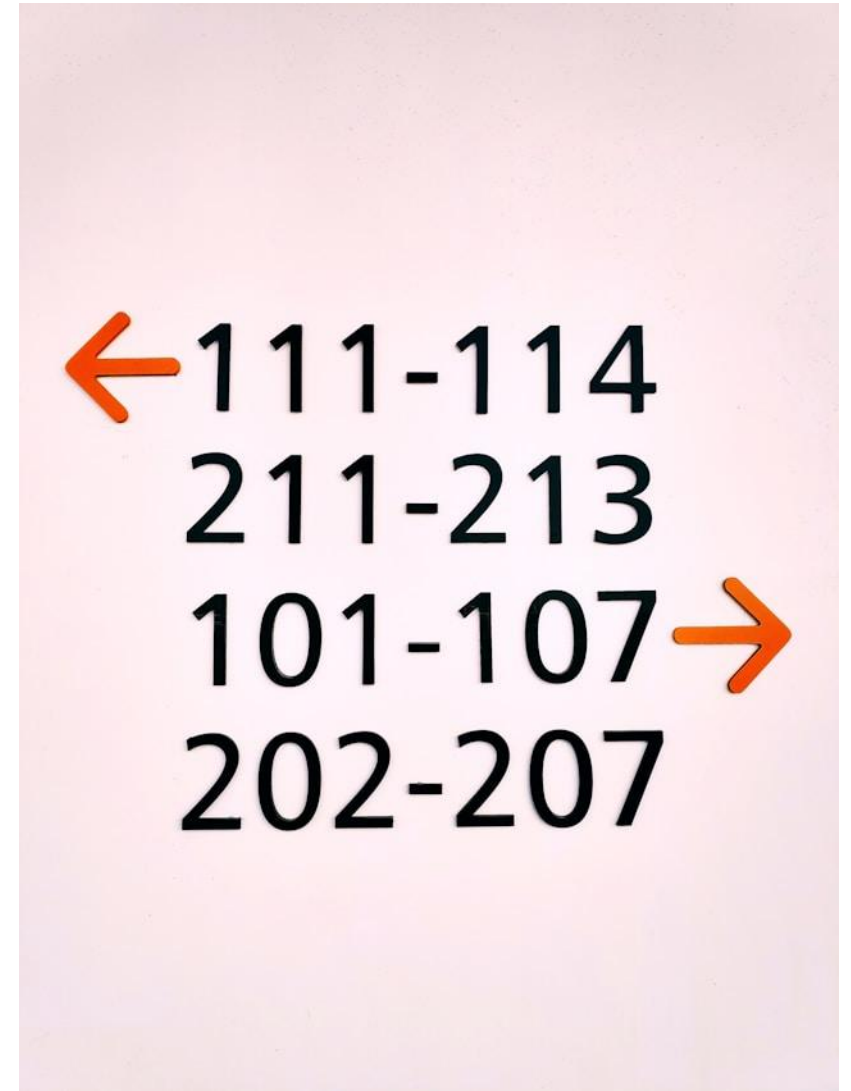
# WSDL Generation/Inspection Tools

- Generic
  - SoapUI, XMLSpy, Oxygen XML Editor
- Language-specific
  - .NET
    - WCF (Windows Communication Foundation), CoreWCF, Visual Studio (WCF Service App)
  - Java
    - Apache CXF, Metro (JAX-WS RI), Spring Web Services, Eclipse IDE Web Services Wizard
  - PHP
    - PHP-SOAP SoapServer class, Zend Framework AutoDiscover class
  - Python
    - Spyne, ZSI



# HTTP Status Codes for SOAP Responses

- HTTP is only SOAP's transport method
  - Semantics are handled directly by SOAP, which supersedes HTTP's status codes
- SOAP 1.1 prescribed returning 500 for all errors (including client errors)
  - Total reliance on WS-Security and Faults
- [SOAP 1.2](#) enforces the use of 4xx status codes
  - 400 for sender errors
    - Fault Sender
  - 500 for receiver errors
    - Fault VersionMismatch, Fault MustUnderstand, FaultReceiver, Fault DataEncodingUnknown



# HTTP Status Codes for SOAP Responses – Examples

- [SOAP Version 1.2 Specification Assertions and Test Collection](#) (Second Edition)
  - [Test TH2](#): the client sends an invalid SOAP request (body missing)
    - The sender is to blame, so the receiver returns a 400 and an env:Sender fault
  - [Test TH5](#): the client requests the media type "audio/mpeg", which is not supported
    - The sender is to blame, so the client returns a 415 Unsupported media
  - [Test TH4](#): the client sends a header with mustUnderstand="1"
    - Since the server cannot understand the said header, the receiver is to blame. The returned status code is 500 Internal Server Error
    - A REST API would surely have returned a 4xx (possibly a 400), but mustUnderstand is a SOAP mechanism, so SOAP handles the semantics directly

# SOAP Implementations – Sorted by Practicality

1. .NET: WCF (Windows Communication Foundation) and CoreWCF
  - Native support for SOAP from the start
2. Java: Apache CXF and Spring Web Services
  - Standards-driven stack (JAX-WS, wsgen, wsimport). Verbose and configuration-heavy
3. PHP: Built-in SoapClient and SoapServer classes
  - Native built-in classes. Fast and lightweight
4. Python: zeep library (client)
  - The spyne library (server) only supports up to Python 3.11. Fragile WSDL generation
5. Go: gowsd1 and gosoap libraries
  - Client only. Can generate client stubs from WSDL, but cannot generate WSDL
6. JavaScript: node-soap and strong-soap
  - Client only. Cannot generate WSDL

# SOAP Enterprise Implementations

- Oracle SOA Suite
- Oracle Service Bus
- SAP NetWeaver PI/PO
- SAP Cloud Integration
- IBM WebSphere
- IBM MQ-based SOA
- Microsoft BizTalk Server
- Mule ESB
- MuleSoft Anypoint

ORACLE



# Sources

- **Cerami, Ethan.** *Web Services Essentials*. O'Reilly, 2002. ISBN: 0-596-00224-6.
- **Chen, Yinnong**, and **Gennaro De Luca.** *Service-Oriented Computing and System Integration: Software, IoT, Big Data, and AI as Services*, 9<sup>th</sup> ed. Kendall Hunt, 2024. ISBN: 979-8-3851-1029-2.
- **Edet, Theophilus.** *Web Services & Service-Oriented Architecture*. CompreQuest, 2023.
- **Lane, Kin.** ["What Is a SOAP API?"](#) Postman Blog, 2023.
- **Newcomer, Eric.** *Understanding Web Services: XML, WSDL, SOAP and UDDI*. Addison Wesley, 2002. ISBN: 978-0-201-75081-2.
- **Samarakkody, Buddhini.** [Jakarta Web Services](#). LinkedIn Learning, 2023.
- Cover picture by [Pixabay](#) from [Pexels](#)